

Chapitre 9 : Introduction au λ -calcul

I Syntaxe du λ -calcul pur

Le λ -calcul est un modèle formel universel de la computation Turing-complet, fondé sur les notions d'abstraction et d'application de fonctions.

A Grammaire formelle des λ -termes

L'ensemble Λ des λ -termes est défini de manière inductive sur un ensemble infini dénombrable de variables $\mathcal{V} = \{x, y, z, \dots\}$ par la grammaire en forme de Backus-Naur (BNF) suivante :

$$t, u ::= x \quad | \quad t u \quad | \quad \lambda x. t$$

- **Variable** : $x \in \mathcal{V}$. Représente un identificateur.
- **Application** : $t u$. Représente l'appel de la fonction t sur l'argument u .
- **Abstraction** : $\lambda x. t$. Représente la définition d'une fonction anonyme prenant le paramètre x et ayant pour corps le terme t .

Méthode : Conventions de notation et associativité

Pour alléger l'écriture des expressions, on applique deux règles syntaxiques strictes :

1. **L'application associe à gauche** : L'expression $t_1 t_2 t_3 \dots t_n$ s'analyse comme $(\dots ((t_1 t_2) t_3) \dots t_n)$.
2. **Le corps d'une abstraction s'étend au maximum vers la droite** : L'expression $\lambda x. t u$ signifie $\lambda x. (t u)$ et non $(\lambda x. t) u$. De plus, les abstractions successives sont contractées : $\lambda x_1 x_2 \dots x_n. t$ abrège $\lambda x_1. \lambda x_2. \dots \lambda x_n. t$.

💡 **Exemple** : Correspondance syntaxique avec OCaml :

- L'identité $\lambda x. x$ s'écrit en OCaml : `fun x -> x.`
- Le duplicateur $\lambda x. x x$ s'écrit en OCaml : `fun x -> x x.`
- L'expression $\lambda x y. x$ s'écrit en OCaml : `fun x -> fun y -> x.`

B Variables libres et variables liées

Une occurrence d'une variable x est dite **liée** si elle se situe dans le sous-terme t d'une abstraction $\lambda x. t$. Dans le cas contraire, elle est dite **libre**.

Définition : Définition inductive de l'ensemble des variables libres $\mathcal{FV}(t)$:

$$\begin{aligned} \mathcal{FV}(x) &= \{x\} \\ \mathcal{FV}(t u) &= \mathcal{FV}(t) \cup \mathcal{FV}(u) \\ \mathcal{FV}(\lambda x. t) &= \mathcal{FV}(t) \setminus \{x\} \end{aligned}$$

🗨 **Vocabulaire** : Un terme t est dit **clos** (ou combinateur) si et seulement si $\mathcal{FV}(t) = \emptyset$.

II Relation de conversion et de réduction

A La α -conversion (renommage)

La sémantique d'une fonction ne dépend pas du choix du nom de son paramètre formel. La α -conversion formalise cette propriété en autorisant le renommage des variables liées sous réserve de ne pas capturer de variables libres existantes.

Définition : On note $t \equiv_{\alpha} u$ la relation d'équivalence engendrée par la règle :

$$\lambda x.t \equiv_{\alpha} \lambda y.(t[x \leftarrow y]) \quad \text{si } y \notin \mathcal{FV}(t)$$

B La Substitution

L'opération fondamentale de calcul repose sur la substitution d'une variable libre par un autre terme. L'opération $t[x \leftarrow u]$ consiste à remplacer toutes les occurrences libres de x dans t par le terme u .

Définition : Définition formelle de la substitution sans capture :

$$\begin{aligned} x[x \leftarrow u] &= u \\ y[x \leftarrow u] &= y && \text{si } y \neq x \\ (t_1 t_2)[x \leftarrow u] &= (t_1[x \leftarrow u]) (t_2[x \leftarrow u]) \\ (\lambda x.t)[x \leftarrow u] &= \lambda x.t \\ (\lambda y.t)[x \leftarrow u] &= \lambda y.(t[x \leftarrow u]) && \text{si } y \neq x \text{ et } y \notin \mathcal{FV}(u) \end{aligned}$$

✗ **Attention** ✗ Si la condition $y \notin \mathcal{FV}(u)$ n'est pas respectée, il y a un risque de **capture de variable**. On doit alors impérativement appliquer une α -conversion préalable sur le terme $\lambda y.t$ pour remplacer y par une variable fraîche $z \notin \mathcal{FV}(u) \cup \mathcal{FV}(t)$ avant d'effectuer la substitution.

C La β -réduction

La β -réduction modélise l'étape élémentaire de calcul par évaluation d'un appel fonctionnel. Un terme de la forme $(\lambda x.t)u$ est appelé un β -redex (réductible expression).

Définition : Règle de β -réduction locale :

$$(\lambda x.t)u \longrightarrow_{\beta} t[x \leftarrow u]$$

On note $\longrightarrow_{\beta}^*$ la fermeture réflexive et transitive de la relation de réduction, représentant une suite d'étapes de calcul. Un terme est dit en **forme normale** s'il ne contient plus aucun β -redex.

III Le λ -calcul simplement typé ($\lambda^{\rightarrow}$)

Le λ -calcul pur permet d'exprimer des termes non terminants (comme $\Omega = (\lambda x.x x)(\lambda x.x x)$). L'introduction d'un système de types permet de restreindre l'ensemble des termes acceptés pour garantir des propriétés comportementales fortes.

A Syntaxe des types

L'ensemble des types simples $\mathcal{T}$ est construit inductivement à partir d'un ensemble de types de base $\mathcal{B}$ (comme `int`, `bool`) et du constructeur de type fonctionnel $\rightarrow$:

$$A, B ::= \iota \quad | \quad A \rightarrow B$$

Le constructeur $\rightarrow$ associe à droite : $A \rightarrow B \rightarrow C$ signifie $A \rightarrow (B \rightarrow C)$.

B Environnement de typage et règles d'inférence

Un **environnement de typage** Γ est un ensemble fini de déclarations de types pour des variables distinctes, noté $\Gamma = \{x_1 : A_1, \dots, x_n : A_n\}$. On écrit $\Gamma, x : A$ pour l'extension de l'environnement Γ avec l'hypothèse que x est de type A , sous réserve que $x \notin \text{dom}(\Gamma)$.

Le jugement de typage s'écrit $\Gamma \vdash t : A$, signifiant « sous l'environnement Γ , le terme t possède le type A ».

Méthode : Règles du système de type de la Dédution Naturelle ($\lambda \rightarrow$)

Les jugements de typage sont dérivés à partir des trois règles logiques suivantes :

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A} \text{ (ax)}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B} (\rightarrow_i) \quad \frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} (\rightarrow_e)$$

💡 **Exemple** : Dérivation de typage pour le terme combinatoire $(\lambda x.x)(\lambda y.y)$ de type $B \rightarrow B$:

$$\frac{\emptyset \vdash \lambda x.x : (B \rightarrow B) \rightarrow (B \rightarrow B) \quad \emptyset \vdash \lambda y.y : B \rightarrow B}{\emptyset \vdash (\lambda x.x)(\lambda y.y) : B \rightarrow B} (\rightarrow_e)$$

Où la sous-dérivation pour l'identité s'obtient par application de la règle d'abstraction :

$$\frac{\{x : B\} \vdash x : B}{\emptyset \vdash \lambda x.x : B \rightarrow B} (\rightarrow_i)$$

C Propriétés fondamentales du système $\lambda \rightarrow$

Le système de types simples valide deux théorèmes métathéoriques majeurs :

Théorème : Subject Reduction

Si $\Gamma \vdash t : A$ et $t \rightarrow_\beta u$, alors $\Gamma \vdash u : A$.

Théorème : Normalisation forte

Si un terme t est typable dans le système $\lambda \rightarrow$ ($\Gamma \vdash t : A$), alors toutes les suites de β -réduction partant de t sont de longueur finie et s'arrêtent sur une forme normale. En conséquence, le terme commun Ω n'est pas typable.

Contents

I	Syntaxe du λ-calcul pur	1
A	Grammaire formelle des λ -termes	1
B	Variables libres et variables liées	1
II	Relation de conversion et de réduction	2
A	La α -conversion (renommage)	2
B	La Substitution	2
C	La β -réduction	2
III	Le λ-calcul simplement typé ($\lambda^{\rightarrow}$)	2
A	Syntaxe des types	2
B	Environnement de typage et règles d'inférence	3
C	Propriétés fondamentales du système $\lambda^{\rightarrow}$	3

Crédits

Ce cours est mis à disposition selon les termes de la Licence Creative Commons  **Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International**. Pour voir une copie de cette licence, visitez <http://creativecommons.org/licenses/by-nc-sa/4.0/>.